

GEMM Working Note #2

deval-d@stanford.edu

May 31, 2026

Designing the short GEMM microkernel

From the previous note, the “short” GEMM routine works on several full columns of C at once. Each column of C is a linear combination of the columns of A , with the weights coming from the corresponding column of B . So, if we update several columns of C together, we can reuse the same pieces of A across several columns of C before moving on.

I previously described the KC blocking as if it directly made the working set small enough for cache. That was not quite right. Splitting the inner dimension into KC-sized chunks only decides how much we accumulate into C before writing it back to memory. It does not, by itself, make data cache-resident. What matters is the data actually touched, and in the microkernel, this is much smaller: we work on a small row chunk of C at a time, keep several columns of that chunk in SIMD registers, and stream through the current KC slice of A and B . So the short kernel still has row blocking, but it happens inside the microkernel. The outer routine chooses a column panel of C and a KC slice of the inner dimension; the microkernel then walks down that column panel in MR-sized row chunks. For each row chunk, it accumulates a small MR×NR block of C in registers.

This note will document the process of designing the microkernel. It accepts a mutable NC-wide column panel of C , a KC-wide column panel of A , and the matching columns of B . Its job is to accumulate

$$C_{\text{panel}} += A_{\text{panel}} B_{\text{block}}.$$

The rest of the short GEMM, then, is roughly:

```
// block sizes to be tuned
const NC: usize = 64;
const KC: usize = 64;

let (m, k_a) = a.dimension();
let (k_b, n) = b.dimension();
let k = {
    assert_eq!(k_a, k_b);
    k_a
};
// dim(C) = (m, n)

let num_nc = n.div_ceil(NC);
let num_kc = k.div_ceil(KC);

for nc_idx in 0..num_nc {
    let j0 = nc_idx * NC;
    let j1 = (j0 + NC).min(n);

    let mut c_panel = c.col_panel_mut(j0..j1);
    let b_panel = b.col_panel(j0..j1);

    // scale C by beta
    for jj in 0..c_panel.n_cols() {
        scal(beta, c_panel.col_mut(jj));
    }

    for kc_idx in 0..num_kc {
        let kc_beg = kc_idx * KC;
        let kc_end = (kc_beg + KC).min(k);

        // C[:, j0..j1] += A[:, kc_beg..kc_end] * B[kc_beg..kc_end, j0..j1]
        microkernel(
            alpha,
            a.col_panel(kc_beg..kc_end),
            b_panel,
            c_panel.reborrow(),
            kc_beg,
        );
    }
}
```

1 Implementation

Full NC-wide contiguous column panels of A and C are used in the microkernel. So for these, the microkernel can just accept a `MatRef` and `MatMut` respectively. These structs are just small containers

```
#[derive(Clone, Copy, Debug)]
pub struct MatRef<'a, T> {
    buffer: &'a [T],
    dimension: (usize, usize),
}
```

physically something like `(ptr, len, nrows, ncols)`. So passing this into a microkernel should not be an issue. It’s just carrying around a reference to the buffer and the dimensions. The annoying case is B . The microkernel uses KC×NC blocks of B , but those blocks are not fully contiguous in memory. Each column only has KC contiguous entries before having to jump over k elements to get to the next block column. So making a clean new `MatRef` for the block of B would require allocating new memory and packing the block.

I think the better thing is to pass in the full contiguous k ×NC column panel of B anyway, and then only access the KC×NC block I actually need. Making the entire panel accessible shouldn’t add any overhead if I don’t actually touch unneeded data. The microkernel just needs one extra `kc_idx` variable so it knows where the current KC-long slice starts inside each column of B .